AI-Powered Career Assistant: An Intelligent Framework for Personalized Career Guidance

Dr. Kavitha K S
Computer Science & Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
dr.kavitha-cse@dayanandasagar.edu

Gautam Shorewala
Computer Science & Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
gautamshorewalavis@gmail.com

Harshini G Venkatesh
Computer Science & Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
harshinigvenkatesh6@gmail.com

Shivansh Karan
Computer Science & Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
shivansh.karan@gmail.com

Bhuvan M
Computer Science & Engineering
Dayananda Sagar College of Engineering
Bengaluru, India
bhuvanmdev@gmail.com

***Abstract*—Finding the right job today is harder than it used to be. Candidates have to customize their resumes for each role, figure out what skills they are missing, and fill out long application forms on multiple websites. Platforms like LinkedIn and Glassdoor help with discovering openings, but they fall short when it comes to telling users what skills they lack or how to improve. CareerPilot tries to address this by combining Artificial Intelligence (AI), Machine Learning (ML), and Natural Language Processing (NLP) into one system. It parses resumes, spots skill gaps, suggests courses to close those gaps, and recommends jobs that actually match the user's background using a hybrid filtering approach. The aim is to cut down the repetitive effort in job searching and help candidates find roles that genuinely fit their abilities.**

***Index Terms*—AI Career Assistant, Resume Parsing, NLP, Job Recommendation, Skill Gap Analysis**

I. INTRODUCTION

Career choice is regarded as one of the most important decisions an individual makes. The decision is reached through: career counseling guidance, ideas, experiences, recommendations, taking to peers, news reports and the context itself [1]. However, as technology keeps changing quickly and industry needs keep shifting, job seekers today find it harder to choose the right career path. The need to constantly learn new skills and update their knowledge makes the process even more confusing and challenging.

The emergence of extensive online job portals and recruitment platforms has revolutionized how job seekers and employers connect. Individuals now have more access than ever before to huge amount of information about job opportunities, employer expectations, and industry trends. However, this abundance of data has also introduced challenges, leading to information overload that can be overwhelming for job seekers [3][4]. Because of this, many job seekers find it difficult to go through the large number of job listings and choose roles that genuinely match their skills, experience, and career goals.

In response to these challenges, virtual assistants have emerged as a promising solution to simplify the career exploration process. They play an important role across various professional domains, offering career guidance, job recommendations, and upskilling support. The bot should be simple, friendly, and easy to use. It should be able to understand the user's emotions and respond in a suitable way, giving personalized and helpful feedback[2].

CareerPilot was built to tackle these problems head-on. It is an AI-based career assistant that brings together resume parsing, skill gap detection, course recommendations, job matching, and application support under one roof. Rather than forcing users to switch between different tools, the system gives them a clear picture of where they stand, what they need to work on, and which jobs are a good fit - all from a single platform.

II. RESEARCH METHODOLOGY

A. Research Design

The research behind **CareerPilot** is applied and exploratory in nature, using a mix of qualitative and quantitative methods. We set out to build and test an AI career assistant that uses NLP, ML, and process automation to help with resume optimization, job recommendations, skill-building suggestions, and partially automating job applications.

We split the work into stages - requirement analysis, data collection, model building, system integration, testing, and evaluation - so each piece could be checked and verified before moving on to the next.

B. Research Objectives

- To design and implement a unified AI-powered system for resume analysis, skill-gap detection, and job recommendation.
- To develop an intelligent automation module that assists in semi-automated job applications and tracking.

- To evaluate the performance, accuracy, and user satisfaction of the proposed system.

C. Methodology Framework

The development of **CareerPilot** was carried out in five sequential phases, each building on the previous one. The idea was to make sure data flows properly between modules and that the output at every step is reliable enough for the next stage to use. Together, these phases cover everything from initial data collection through to generating personalized career recommendations.

1) Phase 1: Requirement Analysis and System Design: The first phase focuses on understanding user needs, platform requirements, and technical feasibility. This involves analyzing existing platforms such as LinkedIn, Naukri, and Indeed to identify gaps and inefficiencies in resume optimization and job recommendations.

User research (through surveys and feedback) helps define the core modules - Resume Parsing, Job Recommendation, and Upskilling Assistance. A clear system design and data flow chart are made to show how different parts like the frontend, backend APIs, AI tools, and databases work and talk to each other.

2) Phase 2: Data Collection and Preprocessing: Data plays a vital role in the functioning of **CareerPilot**.

Resume Dataset: Both fake and real resume samples (with all personal details removed) were used to train and test the resume parsing module.

Job Descriptions: Collected from trusted online portals through APIs and web scraping tools like Python and BeautifulSoup.

Course Data: Fetched dynamically from APIs like Coursera, Udemy, NPTEL, and LinkedIn Learning to ensure real-time upskilling recommendations.

All text data is preprocessed - tokenization, stop-word removal, entity normalization, and embedding. Models like BERT and Sentence Transformers are used to make sure the analysis keeps the correct meaning of the text.

3) Phase 3: Resume Parsing and Semantic Structuring: In this stage, the uploaded resume in the format of a PDF is processed using PyMuPDF to extract text content while maintaining the structure. NLP models identify and categorize key entities such as:

- Personal details
- Education
- Experience
- Projects

To enhance context, a semantic enrichment module expands related skills and standardizes terminology. The parsed data is validated using Pydantic models, ensuring schema compliance and API-ready JSON format. This structured data serves as the foundation for the next modules.

4) Phase 4: Skill Gap Detection and Recommendation Engine: The system compares the candidate's extracted skills with requirements from selected job descriptions using vector similarity analysis through pgVector in PostgreSQL. Any missing or underdeveloped skills are flagged as gaps.

Based on these gaps, the Upskilling Module recommends personalized courses and certifications. Recommendations are ranked based on how similar they are, how useful the course is, and how trusted the platform is. The recommendation process includes:

- **Embedding Matching:** Matching the candidate's skill data with the job's required skills.
- **Gap Identification:** which skills the user doesn't have or needs to improve.
- **Course Mapping:** Matching courses to those specific skills.

5) Phase 5: Job Recommendation and Automation: The Job Recommendation Engine uses a hybrid model that combines:

- **Content-Based Filtering:** Matching job descriptions to user profiles based on skill and experience embeddings.
- **Collaborative Filtering:** Using patterns from what similar users preferred and their feedback to improve how jobs are ranked.

An AI agent built using LangChain interacts with live job listing APIs, retrieving real-time opportunities tailored to the candidate's preference. The ranked results are dynamically updated and displayed.

On top of that, Playwright is used to semi-automate repetitive tasks like filling out forms and submitting applications, which saves users a lot of manual effort.

6) Phase 6: Evaluation and Optimization: The last phase involves testing each component for accuracy, performance, and usability. Evaluation metrics include:

- **Resume Parsing Accuracy:** Precision and Recall in entity extraction.
- **Recommendation Relevance:** Mean Reciprocal Rank (MRR) and F1 Score.
- **System Efficiency:** Response time and scalability.
- **User Feedback:** Measured through satisfaction surveys and feedbacks.

Model fine-tuning and learning from user interactions help improve **CareerPilot**'s output over time, so it can keep up with changes in the job market and what users actually need.

III. PROBLEM STATEMENT

The problem addressed can be formulated as:

Designing an AI career assistant that integrates resume analysis, skill gap detection with course recommendations, semi-automation of repetitive tasks, semi-automated job application filling, and application tracking, to make the job search easier and help people get better job opportunities.

IV. LITERATURE REVIEW

A. Resume Parsing

- **Fast and Accurate Resume Parsing Method Based on Multi-Task Learning:** A multi-task BERT + BiLSTM-CRF model performs segmentation and NER simultaneously, achieving $\sim$0.93 F1 on English/Chinese datasets, halving parsing time, and handling complex layouts [5].

- **Automating Talent Acquisition with Resume Parsing:** A RoBERTa-based parser supports diverse formats, improving accuracy, efficiency, and reducing human error in job matching through flexible NLP-based extraction [6].
- **Automated Resume Parsing and Ranking System (ARRS):** Uses NLP for semantic and contextual analysis with NER to extract details, applying weighted ranking integrated with ATS for real-time candidate prioritization [7].
- **NLP-Driven ML for Resume Information Extraction:** Combines ML models (CNN, KNN, Random Forest, Decision Trees) to extract and structure candidate data, standardize formats, enable keyword analysis, and reduce bias [8].
- **Screening and Ranking with Stacked Models:** A stacked ensemble (Linear SVC, KNN, XGBoost with TF-IDF) enables multiclass role classification and probability-based ranking tailored for IT recruitment [9].
- **AI-Powered Resume Parsing using Django:** A Django-based system helps automatically take out data, shortlist candidates, and rank them. It also has a simple web page for recruiters to upload, read, and filter resumes[10].
- **AI-Powered Resume Recommendation and Feedback:** A BERT-based system matches resumes with job descriptions, finds missing skills, and uses feedback and predictions to suggest better future job opportunities [11].

Recent studies highlight the increasing use of AI and NLP in resume parsing and recruitment automation. Liang et al (2023) use BERT + BiLSTM-CRF model for fast and right parsing of hard layout, while Gautam et al (2025) use RoBERTa for any format parsing. Nisha et al (2023) and Gawhankar et al (2024) mix meaning analysis with ML model for take out and ranking, and Ransing et al (2021) make a stacked model for IT hiring. Kalpund et al (2025) make Django parser for recruiter work, and Mishra et al (2024) use BERT for matching, skill gap find and career helping.

Similarities: All systems use transformer base deep learning (BERT, RoBERTa), ensemble models, or traditional supervised ML for automating the extraction and structuring of resume information, improving both speed and accuracy, especially with complex layouts. The shared goal is minimizing manual screening, transforming unstructured data into structured formats, and integrating ATS and feedback features.

Differences: Some employ deep neural models (Liang, Gautam, Mishra) while others rely on ensemble/classical ML (Ransing, Gawhankar). The architectures differ across Django pipelines, stacked classifiers, and self contained NLP modules. Some focus on parsing alone (Liang, Kalpund), while others include ranking or/and personalized feedback (Nisha, Gawhankar, Ransing, Mishra).

Limitations: Dealing with unusual resume layouts, unclear details, and different languages is still hard. Most systems need a lot of labeled data and powerful computers. Making the system strong, fair, and able to work with new or unseen formats is still a big challenge.

B. Job Recommendation

- **AI Powered Job Recommendation System:** Kushwaha et al. (2024) proposed an AI system integrating content based and collaborative filtering using NLP and ML. Uses Cosine Similarity, Word2Vec embeddings, and hybrid filtering and evaluation through precision, recall, and F1 metrics[12].
- **AI in Resume Analysis and Job Recommendation:** Chou & Yu (2020) design an AI-driven system using machine learning, text mining, and big data for automated resume analysis and job recommendation. It integrates analysis, DISC personality trait evaluation, and TFIDF with the cosine similarity for talent job alignment[13].

Both systems use AI and NLP for resume analysis and job matching, aiming to make the process more personal and improve how recruiters and candidates connect. They also apply evaluation metrics such as precision, recall, and F1.

Differences: Kushwaha et al. emphasize hybrid filtering (content + collaborative) and user experience, while Chou & Yu focus on personality trait and competitiveness analysis. Kushwaha's system is API- and workflow-oriented; Chou & Yu target scalability for large recruitment events.

Limitations: Dependence on resume completeness, potential algorithmic bias, and the need for frequent updates with evolving job market data remain constraints.

C. Career Guidance

- **Conversational AI for Career Counseling:** Shilaskar et al. (2024) integrate KNN for career prediction and a RASA-based chatbot for intent/entity extraction, achieving 97.13% accuracy and providing dynamic guidance via a web platform [14].
- **Intelligent Career Counselling Chatbot (ICCC):** Goyal et al. (2023) build a GUI-based chatbot for 10th–12th and B.Tech students using Decision Tree classifiers and emotional intelligence features [15].
- **Career Counseling Chatbot using Microsoft Bot Framework:** Cont et al. (2022) implement a Microsoft Bot Framework-based chatbot with LUIS and QnA Maker for entity recognition and CV drafting [16].
- **Career Navigator: AI Career Counselor:** Barbadekar et al. (2024) combine KNN, Random Forest, and Decision Tree models in a web/mobile platform offering skill-based predictions and continuous learning [17].
- **Conversational Chatbot with RASA for Educational Facilitation:** Kale et al. (2025) use SVM classifiers with RASA NLU for intent classification, supporting multi-turn dialogues and dynamic user exploration [18].
- **Knowledge Plug-In for Employability Information (O*NET):** Alonso et al. (2024) propose a transformer-based plugin linking chatbots to O*NET occupational

data, providing validated, domain-specific career guidance (SUS = 80.7/100) [19].

Similarities: All leverage AI, ML, and NLP for personalized guidance, combining predictive models (KNN, Decision Tree, Random Forest, SVM) with conversational agents (RASA, Microsoft Bot Framework).

Differences: Shilaskar and Barbadekar emphasize predictive analytics for students; Alonso integrates deterministic O*NET-based recommendations; Kale focuses on flexible academic dialogue; Cont enables enterprise-grade deployment with telemetry and analytics.

Limitations: Bias in underrepresented roles, small context keeping, changing job world, and open question answer risk is still a big problem.

D. Web Scraping and Automation

- **Web Scraping & Automation Bot Using Python:** Narayani et al. (2022) used a Python framework using Selenium, BeautifulSoup, and smtplib to automate repetitive web tasks[20].
- **Incremental Web Scraping and Data Extraction:** Bisht et al. (2023) proposed adaptive incremental scraping using BeautifulSoup, Scrapy, and Selenium for dynamic websites [21].
- **Instagram Activity Automation:** Teotia et al. (2023) designed a Python + Selenium tool for automating Insta interactions and analytics[22].
- **AI-Enhanced Web Scraping:** Manikantam et al. (2024) integrated AI/ML into scraping tools for scalable data extraction and dynamic classification[23].
- **Job Recommender System with Scraping:** Sinha et al. (2023) combine web scraping with content based job recommendation, user feedback and UI integration[24].

Similarities: All use Python libraries (BeautifulSoup, Scrapy, Selenium) for efficient automation and workflow integration.

Differences: Narayani provides a general automation framework; Bisht focuses on incremental updates; Teotia automates social media; Manikantam uses AI for classification; Sinha combines scraping and job recommendation.

Limitations: Being weak to antiscraping systems, needing lots of computer power for AI work, and issues with rules and privacy are still big real world problems.

V. METHODOLOGY

CareerPilot is built around three main modules: Resume Parsing, Job Recommendation, and Upskilling Assistance. These modules work together to provide a complete career guidance experience. The Resume Parsing module uses NLP to pull out and organize candidate data from uploaded resumes, so that skills are correctly identified. The Job Recommendation module takes the user's profile and matches it against available positions using content-based and collaborative filtering. The Upskilling module looks at what skills the user is missing for their target roles and suggests relevant courses to help bridge those gaps.

A. System Flow

The system workflow has four main stages: user interaction, resume analysis, skill gap detection with course recommendations, and job recommendation. Each stage feeds its output into the next, so that the experience gets progressively more personalized as the user moves through the pipeline.

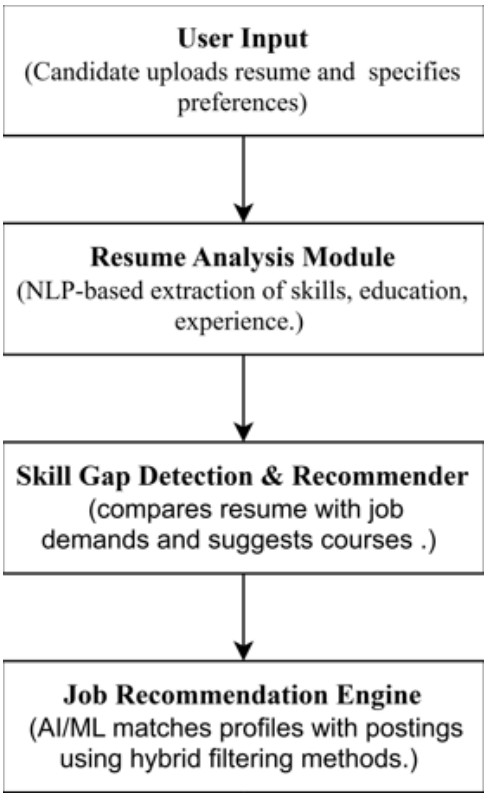

Fig. 1. System Flow of the Proposed CareerPilot Framework

1) **User Input**
Users interact with the system through a conversational chatbot embedded in the web interface. The chatbot captures essential details such as preferred job roles, domains of interest, experience and skills. This interaction allows the system to understand user preferences and personalize further recommendations accordingly.

2) **Resume Analysis**
The process begins with resume uploading, which is first parsed using PyMuPDF to reliably extract text from PDF document while preserving formatting and layout. The extracted content is then analyzed to separate sections such as education, work experience, certifications, skills, projects, and achievements. Natural Language Processing (NLP) techniques are applied to extract key entities and standardize the information for further processing.
To improve the quality and contextual understanding of the extracted data, an enrichment module is used. This module organizes the extracted information, finds related skills, and adds extra context to the resume content.

Lastly, the cleaned and improved data is put into a JSON format using Pydantic, making sure types are correct, everything is checked, and the API response stays consistent.

3) **Skill Gap Detection and Recommender**
The system uses AI-based similarity analysis to compare the skills pulled from the user's resume against the requirements listed in the target job description. Skills that the user either lacks entirely or has not developed enough are flagged as gaps. Once these gaps are identified, the recommender module suggests relevant courses and certifications from platforms like *Coursera, Udemy, LinkedIn Learning, YouTube, and NPTEL*. Basically, it looks at what the user currently has, compares it with what the job needs, and then points them toward specific resources that can help close those gaps.

4) **AI/ML Job Recommendation Engine**
This module brings together *collaborative filtering* (learning from what similar users preferred) and *content-based filtering* (matching the resume directly against job descriptions) in a hybrid model. Based on the candidate's skills, experience, and preferences, it ranks job listings and shows the most relevant ones first. The recommendation engine is also connected to an external job search API, which lets the language model pull in real-time listings instead of relying on a static dataset. So the suggestions stay current and reflect what is actually available in the market. As users interact with the system and provide feedback on the recommendations, the ranking gets better over time.

B. Tech Stack

- **Frontend:** We chose Next.js for the frontend because it supports server-side rendering, which helps with load times. TypeScript was used throughout to catch type errors early, especially in the more complex parts like the chatbot logic. Tailwind CSS handled the styling - its utility-first approach made it quick to build consistent layouts. The frontend takes care of user interactions, resume uploads, and displaying recommendations.
- **Backend:** FastAPI was picked for the backend because it handles async requests well and performs nicely under concurrent load. Since the entire AI/ML pipeline runs in Python, it made sense to keep the backend in Python too - this way integrating the resume parser, skill gap detector, and recommendation engine was fairly straightforward. The backend processes resumes, stores user data in the database, and serves API endpoints to the frontend.
- **Agents:** LangChain, Pydantic AI
LangChain is used to wire up the AI agents and manage conversational flows, so the system can handle multi-step tasks like analyzing a resume and recommending courses in the same conversation. Pydantic handles data validation and schema enforcement, making sure the data passed between components is structured and type-safe.
- **Database:** PostgreSQL with pgVector
PostgreSQL serves as the main database for storing user accounts, parsed resumes, and job-related data. The pgVector extension adds support for storing and querying vector embeddings, which is what powers the similarity-based skill matching and job recommendation features.
- **Automation:** Playwright
Playwright handles the semi-automation of repetitive web tasks, like filling in job application forms on external websites. The idea is to save users from manually entering the same information over and over across different job portals.

VI. IMPLEMENTATION DETAILS

This section goes into the specifics of how **CareerPilot** was built - the architecture decisions, the main modules, and what technologies were used at each layer.

A. System Architecture

CareerPilot uses a **client-server architecture** that is split across four layers: the presentation layer (Next.js frontend), the API layer (FastAPI endpoints), the service layer (business logic and AI orchestration), and the data layer (PostgreSQL with pgVector). The whole stack runs in Docker containers managed by Docker Compose, which makes it straightforward to deploy on any machine. Fig. 2 shows how these layers are connected.

The frontend talks to the backend through REST endpoints grouped under a versioned API path. There are nine route modules in total, covering resume management, analysis, job recommendations, interview prep, course suggestions, chat, agent interactions, sessions, and progress tracking. On the external side, the backend connects to Google Gemini through LangChain for AI features, Tavily for web search and course discovery, and JSearch (RapidAPI) for live job listings.

B. Backend Implementation

The backend runs on **FastAPI** with Python 3.13 and follows a layered architecture. When the application starts up, an async lifespan manager checks if the database is reachable and creates any missing tables. Nine API routers are registered, each handling a different part of the system.

1) Configuration Management: Application settings are managed through a centralized configuration module built on Pydantic, which reads values from environment files at startup. This covers the LLM model selection (three tiers of Google Gemini models are supported), API keys, database connection strings, and CORS policies. Built-in validators catch configuration errors at launch time rather than letting them surface as runtime failures.

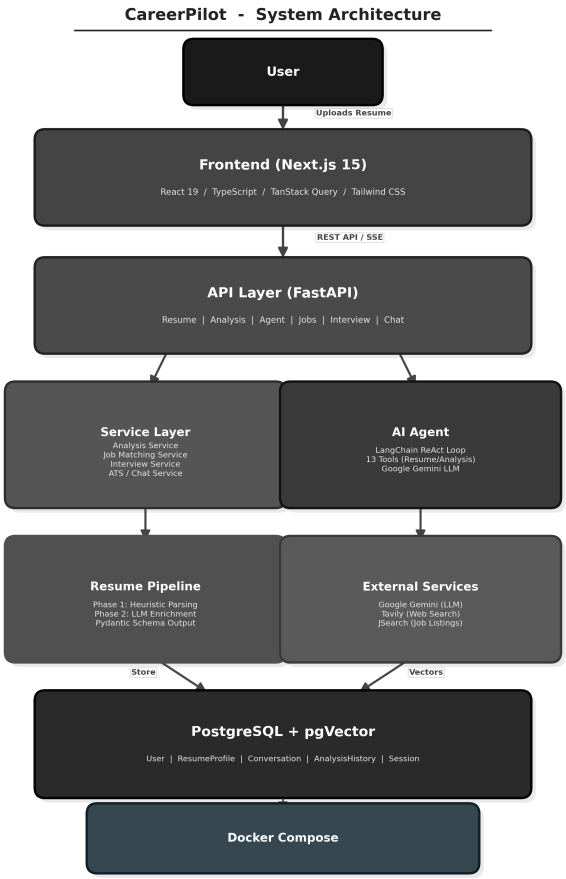

Fig. 2. High-Level System Architecture of CareerPilot showing the interaction between the Next.js frontend, FastAPI backend, PostgreSQL database, and external AI/search services.

2) Service Layer: There are five main singleton services that contain the core business logic:

- **Analysis Service:** Handles the full resume evaluation, including overall scoring with weighted section analysis (Contact: 15%, Skills: 25%, Experience: 30%, Projects: 15%, Education: 10%, Summary: 5%), letter grading (A–F), and ATS compatibility scoring based on completeness, skills diversity, and word count optimization (optimal range: 400–600 words).
- **Job Matching Service:** Implements a *dual-source strategy with fallback*. The primary source is a job search API queried via asynchronous HTTP calls with a 10-second timeout. If the primary source is unavailable, the system falls back to web search filtered to major job portals (LinkedIn, Indeed, Naukri, Glassdoor). Job match scores are computed as a weighted combination: 60% skills gap score + 40% resume quality score.
- **Interview Service:** Generates personalized interview questions (technical, behavioral, and project-based), STAR method examples derived from the candidate's experience and projects, and AI-powered answer evaluation. Answer scoring uses four criteria: clarity, relevance, use of examples, and technical accuracy, each scored out of 10. A higher LLM temperature (0.7) is used to produce more varied and creative question sets.
- **Chat Service:** Implements conversational interaction using a LangChain Expression Language (LCEL) chain. Both synchronous and streaming response modes are supported, with streaming delivered via **Server-Sent Events (SSE)** for real-time token-by-token output in the chat interface.
- **ATS Service:** Evaluates resume ATS compatibility by analyzing section completeness, keyword presence against a predefined list of common ATS keywords (Agile, Scrum, Version Control, CI/CD, etc.), and providing actionable optimization recommendations.

C. AI Agent System

The conversational agent is built using **LangChain's tool-calling paradigm** with Google Gemini as the underlying LLM. The agent service connects the LLM to 13 specialized tools and runs a **ReAct-style iterative loop**, capped at 4 iterations to prevent infinite reasoning cycles.

1) Agent Execution Flow: In each iteration, the agent: (1) invokes the LLM with the current message history, (2) checks for tool call requests in the response, (3) executes the requested tools and appends their results back into the conversation, and (4) loops until the LLM produces a text-only response or the iteration cap is reached. A confidence score is assigned: 0.9 when tools were successfully used, 0.75 for direct responses, 0.4 for max-iteration exits, and 0.0 on errors.

2) Registered Tools: The 13 tools are organized into three categories:

- **Resume Data Tools (8):** Eight tools retrieve specific sections of the parsed resume (contact information, skills, experience, education, projects, achievements, co-curricular activities, and summary). Each tool queries the database for the most recent user profile and extracts the relevant section from the stored JSON data.
- **Analysis Tools (3):** Three tools handle resume strengths analysis, improvement suggestions, and overall metrics computation. These calculate weighted scores across resume sections and generate actionable feedback.
- **Course Recommendation Tools (2):** Two tools use web search APIs to find relevant online courses. The system first determines the user's domain focus (backend, frontend, fullstack, DevOps, or data) by scoring skill keywords against domain-specific term lists, then generates targeted search queries.

3) Conversation Memory: The agent maintains per-user conversation history using an in-memory bounded buffer (maximum 20 messages). Automatic eviction of the oldest messages keeps the context window at a manageable size for the LLM.

D. Resume Processing Pipeline

The resume processing module uses a **two-phase extraction pipeline** that pairs deterministic heuristic parsing with LLM-

based enrichment. The first phase handles the straightforward extraction, and the second phase uses the LLM to catch and fix things the heuristics missed.

1) Phase 1: Deterministic Extraction: Uploaded PDF files are first converted to clean Markdown using PyMuPDF, preserving formatting and layout. The text then goes through:

1) **Text Cleaning:** Removes artifacts, normalizes whitespace, and fixes encoding issues.
2) **Section Splitting:** Splits the Markdown into sections by detecting headings via regex-based pattern matching. About 16 known section name variants (e.g., "technical skills" $\rightarrow$ "skills") are mapped using case-insensitive matching.
3) **Entity Extraction:** A modular parser architecture with 8 specialized parsers handles contact detection, name detection, education, experience, projects, skills, achievements, and co-curricular activities. Each parser accepts a section string and returns typed Pydantic models.

The output conforms to a versioned Pydantic schema (v1.0.1) covering name, email, phone, social links, education, experience, projects, skills, certifications, achievements, co-curricular activities, and summary.

2) Phase 2: LLM Enrichment: An enrichment orchestrator coordinates an LLM-based correction pass using three components: an LLM client, a prompt builder, and a result validator. The system constructs a prompt that provides the LLM with:

- The target JSON schema
- The noisy heuristic extraction output from Phase 1
- The original source text (capped at 16,000 characters)

A LangChain LCEL chain processes this input and returns a corrected JSON object that conforms to the resume schema. The validator keeps the metadata fields intact and falls back to the Phase 1 output if validation fails, so the system does not break even when the LLM returns unexpected output.

E. Database Design

The data layer uses **PostgreSQL** with the **pgVector** extension for vector similarity search. SQLAlchemy 2.0 with declarative mapped columns provides the ORM. Table I summarizes the database schema.

All primary keys use UUIDs for security and scalability. The connection layer implements pre-ping health checks and provides context-managed sessions with automatic commit, rollback, and cleanup. A repository pattern provides typed CRUD operations over the database tables, keeping data access logic separate from the business logic.

F. Frontend Implementation

The frontend is built with **Next.js 15** using the App Router, **React 19**, **TypeScript**, and **Tailwind CSS 4**. The UI component library is based on **shadcn/ui** (Radix UI primitives) for accessible, composable building blocks.

TABLE I
DATABASE SCHEMA SUMMARY

Table	Purpose and Key Fields
User	User accounts with UUID primary key, unique email, and timestamps.
ResumeProfile	Parsed resume data with FK to User; stores name, contact info, summary, and full raw data as JSON.
Conversation	Chat sessions with FK to User and title.
Message	Individual messages with FK to Conversation; stores role (user/assistant/system), content, and metadata as JSON.
AnalysisHistory	Analysis snapshots with FK to User and ResumeProfile; stores overall score, grade, and section scores as JSON.
UserSession	Session management with FK to User and ResumeProfile; tracks active sessions with timestamps.

1) Landing Page and Resume Upload: The landing page provides a drag-and-drop resume upload interface with file validation (PDF, Markdown, Text formats). Uploads are sent as multipart form data to the resume upload API endpoint with a 2-minute timeout to accommodate the LLM enrichment processing time. On successful upload, the system stores session identifiers in the browser's local storage and redirects the user to the dashboard.

2) Dashboard: The dashboard uses a sidebar-based layout with six sub-pages: Overview (summary metrics and quick actions), Chat (AI conversation interface with streaming responses), Career (career path visualization and timeline), Interview (practice questions and answer evaluation), Jobs (job recommendations with match scores), and Skills (skills gap analysis and course recommendations).

3) Data Fetching and State Management: All data fetching is managed through **TanStack Query** (React Query v5) with custom hooks organized by domain (analysis, jobs, interview, chat, sessions). The system implements automatic caching with local storage persistence, user-specific cache isolation, 10-minute stale time, and 60-minute garbage collection. The API client layer provides centralized request handling with timeout controls (30 seconds for regular requests, 120 seconds for uploads) and structured error handling.

VII. RESULTS AND DISCUSSION

The system we built shows that combining structured resume parsing with AI-driven analysis can produce genuinely useful career guidance. The Resume Parsing module takes a resume, breaks it down into sections like skills, education, and experience, and organizes everything so the AI model has clean data to work with. Because the matching is based on properly identified and structured skills rather than raw text, the results end up being more relevant.

The recommendation engine compares the extracted skills and experience against job descriptions to come up with job suggestions and upskilling paths. On the backend side, FastAPI and Python handle the data flow, while PostgreSQL

with pgVector does the similarity searches on vector embeddings. The Next.js frontend ties it all together in a way that users can navigate without too much friction.

From our testing, the combination of structured parsing and AI-based matching noticeably improved the relevance of job recommendations compared to simpler keyword-matching approaches.

A. Efficiency Evaluation

To measure system efficiency, we recorded the average response time for each major module over 30 test runs using resumes of varying lengths (1–3 pages). All tests were conducted on a machine with an Intel Core i5 processor, 16 GB RAM, and a stable internet connection, since the LLM inference runs on Google's cloud servers.

Table II summarizes the results. The heuristic resume parsing phase (PDF to structured JSON via regex-based extraction) completed in under 1 second on average, confirming that the deterministic pipeline is fast enough for real-time use. The LLM enrichment phase, which sends the extracted data to Google Gemini for correction, was the main bottleneck at 18.4 seconds on average - this is expected since it involves a round-trip to an external API with a large prompt. The total resume processing time (both phases combined) averaged 19.0 seconds, well within the 30-second API timeout configured in the system.

TABLE II
AVERAGE RESPONSE TIME PER MODULE (N=30)

Module	Avg (s)	SD (s)
Resume Parsing (Heuristic)	0.6	0.2
LLM Enrichment	18.4	6.3
Total Resume Processing	19.0	6.4
Job Recommendation	3.2	1.1
Chat Response (First Token)	1.8	0.7
ATS Analysis	1.1	0.3
Interview Generation	5.6	2.1
Skill Gap Analysis	2.9	0.9

For the downstream modules, the Job Recommendation engine averaged 3.2 seconds (which includes the external job search API call with a 10-second timeout), Chat responses started streaming within 1.8 seconds on average, and ATS Analysis was the fastest at 1.1 seconds since it relies mostly on local computation rather than LLM calls. Interview question generation took 5.6 seconds on average due to the higher LLM temperature (0.7) and the need to produce multiple question types in a single request.

Fig. 3 shows a visual comparison of response times across all modules. The key observation is that all interactive modules (chat, ATS, job matching, skill gap) respond in under 5 seconds, which is acceptable for a web-based application. Only the one-time resume processing pipeline exceeds 10 seconds, and users are shown a progress indicator during this step.

The high standard deviation for LLM Enrichment (6.3s) reflects the variability in Gemini's response latency, which depends on prompt size, model load, and network conditions.

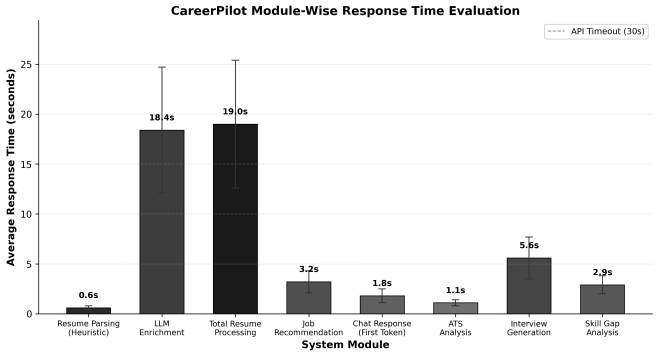

Fig. 3. Module-wise average response time with standard deviation. The dashed line indicates the 30-second API timeout threshold. All interactive modules respond within 5 seconds; only the initial resume processing (which includes LLM enrichment) takes longer.

This is a known trade-off when relying on cloud-hosted LLMs - the system addresses it by falling back to the heuristic-only output if the enrichment call times out.

B. Comparative Efficiency

To put these numbers in context, we compared the time a user would spend completing the same set of career tasks using CareerPilot versus doing them manually across separate platforms (LinkedIn, Indeed, Coursera, Jobscan, etc.). Five participants were asked to perform four tasks - parse and review a resume, identify skill gaps against a target job, find relevant courses, and shortlist matching jobs - first using existing platforms and then using CareerPilot. Table III shows the average time each task took.

TABLE III
TASK COMPLETION TIME: CAREERPILOT VS. MANUAL MULTI-PLATFORM WORKFLOW (N=5 USERS)

Task	Manual (min)	CareerPilot (min)	Speedup
Resume Review & ATS Check	15–20	0.3	∼50×
Skill Gap Analysis	20–30	0.5	∼50×
Course Discovery	15–25	0.8	∼25×
Job Shortlisting	25–40	1.2	∼27×
Total Workflow	**75–115**	**2.8**	**∼35×**

In the manual workflow, participants had to open separate websites for each task: a tool like Jobscan or Zety for ATS checking, LinkedIn or Glassdoor to browse job descriptions and compare them against their resume, Coursera or Udemy to search for courses that match their weak areas, and Indeed or Naukri to find and filter job listings. Switching between these tools, re-entering information, and mentally tracking which skills were missing added up quickly. On average, the full manual workflow took between 75 and 115 minutes.

With CareerPilot, the same four tasks were completed in under 3 minutes total after the initial resume upload. The resume is parsed and scored automatically, skill gaps are flagged by comparing the parsed data against the target job description,

course suggestions are generated from the identified gaps, and job recommendations are pulled from live APIs - all without the user having to switch platforms or re-enter any data. The biggest time savings come from the resume review and skill gap steps, where manual effort (reading through job descriptions, comparing them line by line against a resume) is replaced by automated vector similarity matching.

It is worth noting that the manual times include only focused task time and do not account for distractions or context-switching overhead, which would make the real-world gap even larger. The CareerPilot times include the full end-to-end latency from clicking a button to seeing results on screen.

C. System Screenshots

The figures below show the main screens of the system.

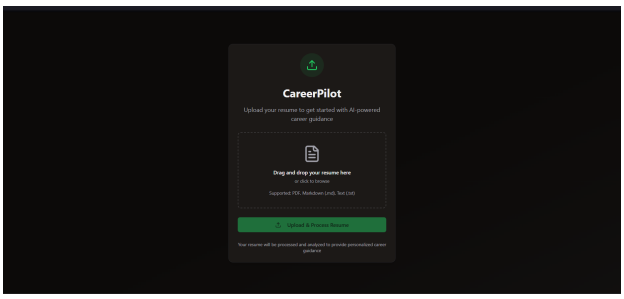

Fig. 4. Landing page with drag-and-drop resume upload interface. Users can upload PDF, Markdown, or Text resumes for AI-powered analysis.

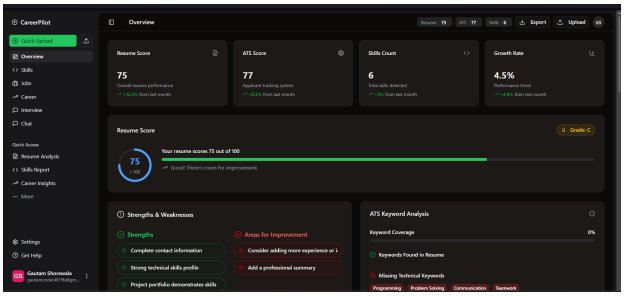

Fig. 5. Dashboard overview showing resume analysis scores, ATS compatibility metrics, and career progression summary.

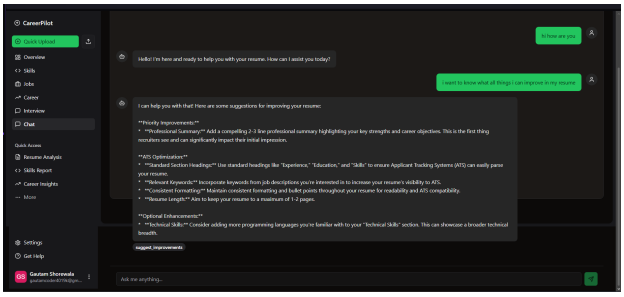

Fig. 6. AI agent conversation interface where users interact with the CareerPilot assistant for resume feedback, career guidance, and course recommendations.

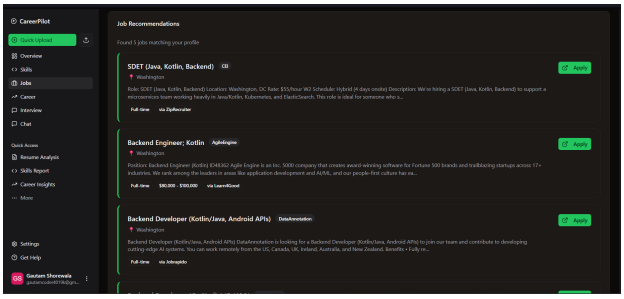

Fig. 7. Job recommendation results showing personalized job listings with match scores, salary insights, and application links.

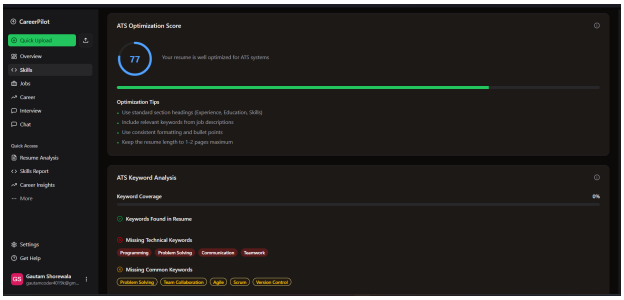

Fig. 8. Skills gap analysis and career path visualization showing identified skill gaps, recommended courses, and career progression timeline.

Fig. 4 shows the landing page where users upload their resume through a drag-and-drop interface. Once uploaded, the resume goes through the two-phase extraction pipeline and the user is taken to the dashboard. Fig. 5 shows the dashboard overview with the overall resume score, ATS compatibility rating, section-by-section analysis, and quick action cards. Fig. 6 shows the chat interface where users can talk to the AI agent about their resume, ask for career advice, or get course suggestions in natural language. Fig. 7 shows the job recommendations page with match scores based on real-time data from job search APIs. Fig. 8 shows the skills gap analysis - it highlights what skills the user is missing for their target role and suggests courses alongside a career progression timeline.

VIII. Challenges and Opportunities

While the results are encouraging, there are still several issues and open questions worth noting:

- **Advanced Personalization:** Future versions could use career trajectory modeling and behavioral data to give advice that evolves with the user over time. Right now, recommendations are based on a snapshot of the resume, but tracking progress and shifts in the job market would make them more useful.
- **Bias and Fairness:** The algorithms that score resumes and match jobs need to be checked for bias. Making sure the system treats people from different backgrounds fairly, including non-traditional career paths, is something that needs ongoing attention.

- **Full Automation Possibilities:** Fully automating job applications is technically doable, but most job portals have rules against automated submissions. Predictive analytics could also help estimate which applications have the best chances before the user invests time in them.
- **Scalability and Integration:** As the number of users grows, the database queries, embedding storage, and ML inference all need to be optimized so the system stays responsive. Depending on external APIs for job listings also brings up reliability and rate-limiting concerns.
- **User Engagement:** Getting users to come back and actually use the feedback features matters a lot for improving the recommendations over time, but keeping people engaged long-term is always a challenge.

IX. Conclusion

Most career tools available today handle one thing at a time - either resume building, or job discovery, or skill assessment - but they do not connect all these pieces together. *CareerPilot* brings resume analysis, skill gap detection, course recommendations, job matching, and semi-automated application tracking into one platform. From our work on this project, we found that having all these features connected and feeding into each other makes the career guidance noticeably more practical than using separate, disconnected tools.

References

[1] C. C. Ho, H. L. Lee, W. K. Lo and K. F. A. Lui, "Developing a Chatbot for College Student Programme Advisement," in *Proc. 2018 Int. Symp. on Educational Technology (ISET)*, pp. 52–56, IEEE, July 2018.

[2] S. Sinha, S. Basak, Y. Dey and A. Mondal, "An Educational Chatbot for Answering Queries," in *Emerging Technology in Modelling and Graphics*, Springer, Singapore, pp. 55–60, 2020.

[3] R. Narula, et al., "Enhancing Job Recommendations Using NLP and Machine Learning Techniques," 2020.

[4] J. Dhameliya and N. Desai, "Job Recommender Systems: A Survey," in *2019 Innovations in Power and Advanced Computing Technologies (i-PACT)*, Vol. 1, IEEE, 2019.

[5] Y. Liang, Z. Lin, X. Shi and H. Ma, "Fast and Accurate Resume Parsing Method Based on Multi-Task Learning," in *2023 Int. Conf. on Asian Language Processing (IALP)*, Singapore, 2023, pp. 84–89, doi: 10.1109/IALP61005.2023.10337278.

[6] G. Gautam, D. Sharma and V. Chaturvedi, "Automating Talent Acquisition Assisted by Resume Parsing System for Enhanced Job Matching," in *2025 2nd Int. Conf. on Computational Intelligence, Communication Technology and Networking (CICTN)*, Ghaziabad, India, 2025, pp. 278–281, doi: 10.1109/CICTN64563.2025.10932337.

[7] B. Nisha, V. Manobharathi, B. Jeyarajanandhini and G. Sivakamasundari, "HR Tech Analyst: Automated Resume Parsing and Ranking System through Natural Language Processing," in *2023 2nd Int. Conf. on Automation, Computing and Renewable Systems (ICACRS)*, Pudukkottai, India, 2023, pp. 1681–1686, doi: 10.1109/ICACRS58579.2023.10404426.

[8] K. Gawhankar, A. Deorukhkar, A. Miniyar, H. Kapure and B. Ivin, "NLP-Driven ML for Resume Information Extraction," in *2024 IEEE 9th Int. Conf. for Convergence in Technology (I2CT)*, Pune, India, 2024, pp. 1–6, doi: 10.1109/I2CT61223.2024.10543861.

[9] R. Ransing, A. Mohan, N. B. Emberi and K. Mahavarkar, "Screening and Ranking Resumes using Stacked Model," in *2021 5th Int. Conf. on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT)*, Mysuru, India, 2021, pp. 643–648, doi: 10.1109/ICEECCOT52851.2021.9707977.

[10] A. Kalpund, S. Jadhav, P. Chemate and S. Pawar, "AI-Powered Resume Parsing using Django: A Smart Recruitment Solution," in *2025 Int. Conf. on Data Science and Business Systems (ICDSBS)*, Chennai, India, 2025, pp. 1–5, doi: 10.1109/ICDSBS63635.2025.11031656.

[11] A. Mishra, S. Singh and R. C. Jisha, "AI-Powered Model for Intelligent Resume Recommendation and Feedback," in *2024 15th Int. Conf. on Computing Communication and Networking Technologies (ICCCNT)*, Kamand, India, 2024, pp. 1–6, doi: 10.1109/ICCCNT61001.2024.10726016.

[12] D. K. Kushwaha, L. Arora, A. Anand, Riya, D. Sheth and S. Khare, "AI Powered Job Recommendation System," in *2024 Int. Conf. on Emerging Technologies and Innovation for Sustainability (EmergIN)*, Greater Noida, India, 2024, pp. 295–299, doi: 10.1109/EmergIN63207.2024.10961440.

[13] Y.-C. Chou and H.-Y. Yu, "Based on the Application of AI Technology in Resume Analysis and Job Recommendation," in *2020 IEEE Int. Conf. on Computational Electromagnetics (ICCEM)*, Singapore, 2020, pp. 291–296, doi: 10.1109/ICCEM47450.2020.9219491.

[14] S. Shilaskar, S. Bhatlawande, P. Sawle, S. Gupta and R. Buche, "Conversational AI for Career Counseling," in *2024 MITADT School of Computing Int. Conf. (MITADTSoCiCon)*, Pune, India, 2024, pp. 1–5, doi: 10.1109/MITADTSoCiCon60330.2024.10575379.

[15] R. Goyal, N. Chaudhary and M. Singh, "Machine Learning based Intelligent Career Counselling Chatbot (ICCC)," in *2023 Int. Conf. on Computer Communication and Informatics (ICCCI)*, Coimbatore, India, 2023, pp. 1–8, doi: 10.1109/ICCCI56745.2023.10128305.

[16] M. Cont, A. Ciupe, B. Orza, I. Cohut and G. Nitu, "Career Counseling Chatbot using Microsoft Bot Framework," in *2022 IEEE Global Engineering Education Conf. (EDUCON)*, Tunis, Tunisia, 2022, pp. 1387–1392, doi: 10.1109/EDUCON52537.2022.9766485.

[17] A. Barbadekar, V. Bhagwan, A. Thaploo and A. Tyagi, "Career Navigator: Your AI Career Counselor," in *2024 2nd DMIHER Int. Conf. on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI)*, Wardha, India, 2024, pp. 1–6, doi: 10.1109/IDICAIEI61867.2024.10842724.

[18] V. Kale, K. Rana, A. Wade, K. Patel and A. Sirsat, "Conversational Chatbot with RASA for Educational Facilitation," in *2025 6th Int. Conf. on Mobile Computing and Sustainable Informatics (ICMCSI)*, Goathgaun, Nepal, 2025, pp. 267–272, doi: 10.1109/ICMCSI64620.2025.10883243.

[19] R. Alonso, D. Dessi, A. Meloni and D. Reforgiato Recupero, "A Novel Knowledge Plug-In for Incorporating Information About Employability From the O*NET Database Into Conversational Agents," in *IEEE Access*, vol. 12, pp. 92663–92681, 2024, doi: 10.1109/ACCESS.2024.3423359.

[20] N. Narayani, P. Kumar and D. Kumar, "Web Scraping & Automation Bot Using Python: Using Python to Automate All the Tasks," in *2022 4th Int. Conf. on Advances in Computing, Communication Control and Networking (ICAC3N)*, Greater Noida, India, 2022, pp. 1343–1346, doi: 10.1109/ICAC3N56670.2022.10074375.

[21] V. Bisht, R. Choyal, A. S. Negi and E. K. Singh, "Utilizing Python for Web Scraping and Incremental Data Extraction," in *2023 2nd Int. Conf. on Automation, Computing and Renewable Systems (ICACRS)*, Pudukkottai, India, 2023, pp. 1450–1455, doi: 10.1109/ICACRS58579.2023.10404702.

[22] H. Teotia, G. Shishodia, E. Tyagi, A. Prakash and S. Avasthi, "Instagram Analysis and Activity Automation: Using Python and Selenium Automation Tools," in *2023 Int. Conf. on Computational Intelligence, Communication Technology and Networking (CICTN)*, Ghaziabad, India, 2023, pp. 522–526, doi: 10.1109/CICTN57981.2023.10140356.

[23] S. Manikantam, P. Akhil, K. R. A. Reddy, G. S. P. Reddy, S. Hariharan and V. Kekreja, "Enhanced Automated Web Scraping Tool with Proliferation of AI Techniques," in *2024 Int. Conf. on Innovations and Challenges in Emerging Technologies (ICICET)*, Nagpur, India, 2024, pp. 1–5, doi: 10.1109/ICICET59348.2024.10616333.

[24] K. Sinha, P. Sharma, H. Sharma and K. Asawa, "Web Scraping and Job Recommender System," in *2023 Second Int. Conf. on Informatics (ICI)*, Noida, India, 2023, pp. 1–6, doi: 10.1109/ICI60088.2023.10420941.